

SQLite-Carver-Pro v1.7.0

Manuel de Référence Technique & Spécifications Internes d'Ingénierie Forensique

 **Analyse Physique des B-Trees, Carving de Slack & Rétro-Ingénierie Forensique**

Documentation approfondie décrivant les mécanismes physiques de stockage, les algorithmes de récupération de cellules supprimées, la détection de stéganographie par page, la reconstruction de transactions WAL et la résurrection de schémas de bases de données SQLite sans interpréteur SQL.

Auteur : Dam-FOR3K

Version : 1.7.0 (Édition Professionnelle Forensique)

Architecture Cible : Moteur Autonome Air-Gapped (Windows x86_64, Linux, macOS)

Conformité Méthodologique : ISO/IEC 27037 (Collecte et Préservation de Preuves Numériques)

1. Anatomie Physique d'une Base SQLite & Zones d'Audit

Contrairement aux outils de gestion de bases de données traditionnels qui s'appuient sur l'API officielle SQLite (`sqlite3.dll`), **SQLite-Carver-Pro** opère directement au niveau binaire sur les pages physiques brutes du fichier. Cette approche permet de s'affranchir des limitations du moteur de requêtes qui ignore systématiquement les données désallouées, tronquées ou corrompues.

1.1 L'En-Tête de Base de Données (100 Octets)

Les 100 premiers octets du fichier constituent l'en-tête de base de données (présent uniquement sur la Page 1). Chaque champ fournit des métriques critiques pour l'analyse judiciaire :

Offset	Taille	Nom du Champ	Description & Rôle Forensique
0	16 B	Magic Header String	Chaîne ASCII obligatoire : <code>SQLite format 3</code> . Valide l'intégrité de signature.
16	2 B	Page Size	Taille physique des pages (512 à 65536 octets, puissance de 2). Conditionne tous les calculs d'offset.
18	1 B / 1 B	Write / Read Version	Version de format de fichier (1 = Rollback Journal conventionnel, 2 = Write-Ahead Log / WAL).
20	1 B	Page Reserved Space	Zone critique : Nombre d'octets réservés à la fin de chaque page (0 à 255). Ces octets sont complètement ignorés par le moteur B-Tree et ne sont jamais écrasés par des cellules. Ils constituent un vecteur majeur de dissimulation stéganographique et d'exfiltration anti-forensique.
24	4 B	File Change Counter	Incrémenté à chaque transaction d'écriture. Permet de détecter les falsifications ou restaurations non autorisées.
28	4 B	Database Size in Pages	Nombre de pages valides. Permet d'identifier immédiatement le <i>Slack EOF</i> (données cachées en fin de fichier).
32	4 B	First Freelist Trunk Page	Pointeur vers la première page tronc de la liste des pages libérées (freelist).
36	4 B	Total Freelist Pages	Nombre total de pages libérées et réutilisables dans la base de données.
56	4 B	Text Encoding	Encodage textuel par défaut (1 = UTF-8, 2 = UTF-16le, 3 = UTF-16be).

1.2 Typologie des Pages B-Tree

Chaque page de la base de données est structurée selon l'un des 4 types B-Tree définis par son premier octet :

- **Table Leaf Page (0x0D)** : Contient les données applicatives réelles (cellules de données) et le RowID (clé primaire entière). C'est la source principale d'extraction.
- **Table Interior Page (0x05)** : Arbre de routage des tables contenant les clés RowID et les pointeurs de pages filles vers d'autres pages B-Tree.

- **Index Leaf Page (0x0A)** : Contient les clés d'index et le RowID cible. Est également utilisée pour le stockage des tables `WITHOUT ROWID` où la totalité des colonnes réside dans l'index.
- **Index Interior Page (0x02)** : Arbre de navigation des index pointant vers les pages feuilles d'index.

1.3 Compartiments Physiques d'une Page & Slack Forensique

Une page SQLite alloue son espace physique de haut en bas et de bas en haut, créant plusieurs compartiments distincts :

Compartiment	Sens d'Allocation	Contenu & Signification Forensique
Page Header	Début de page (Offset 0 ou 100)	Définit le type B-Tree, le premier freeblock (offset 1), le nombre de cellules (offset 3), et le début de la zone de contenu (offset 5).
Cell Pointer Array	Descendant (vers le bas)	Tableau d'entiers 16 bits non signés (big-endian) pointant vers le début de chaque cellule active.
Unallocated Space	Intervalle libre médian	Espace physique situé entre la fin du tableau de pointeurs et le début de la zone de contenu de cellules. Recèle les cellules orphelines après déréférencement.
Cell Content Area	Ascendant (du bas vers le haut)	Zone où sont écrites les cellules actives. Les nouvelles cellules sont allouées en remontant vers le centre de la page.
Freeblocks	Dispersés dans la zone de contenu	Cellules supprimées chaînées entre elles via une liste simplement liée. Contiennent les corps de données effacées récemment.
Fragmented Free Slack	Octets isolés (Offset 7 du header)	Résidus d'espaces de moins de 4 octets créés lors de micro-réallocations.
Page Reserved Space	Pied de page (Offset : PageSize - Reserved)	Zone réservée invariable située en toute fin de page, inviolable par le moteur B-Tree.

2. Algorithmes de Carving & Rétro-Ingénierie Forensique

Cette section détaille les mécanismes internes développés dans SQLite-Carver-Pro pour extraire, requalifier et reconstituer les données supprimées ou dissimulées.

2.1 Carving des Freeblocks & Reconstruction d'En-Têtes Détruits

Pourquoi : Lorsqu'un enregistrement est supprimé, SQLite ne supprime pas immédiatement ses données. Si l'espace libéré mesure au moins 4 octets, SQLite le transforme en *freeblock* et l'insère dans une liste simplement liée ordonnée par offset croissant, dont la tête se trouve à l'offset 1 du header de page.

Le Problème Technique : Pour maintenir la liste chaînée, SQLite écrase systématiquement les 4 premiers octets de l'ancienne cellule par :

```
[ Pointeur vers le prochain Freeblock (2 octets, uint16_be) ] [ Taille du Freeblock (2 octets, uint16_be)
```

Cet écrasement détruit l'en-tête natif de la cellule (le varint de payload size et le varint de RowID), rendant la cellule illisible pour les parseurs naïfs.

La Solution Implémentée : SQLite-Carver-Pro isole la zone préservée (à partir du 4ème octet du freeblock) et applique une évaluation dynamique de séquences candidates (`decode_truncated_record`) :

1. Recherche de signatures valides de *Record Header* (suite de types série cohérents).
2. Calcul de la longueur totale attendue du corps : `L_corps =  $\sum$  serial_type_length(st)`.
3. Validation de l'invariant de frontière : l'espace restant dans le freeblock doit concorder exactement avec la somme des longueurs de types, restaurant la structure sans faux positif.

2.2 Carving de l'Espace Non-Alloué & Fenêtre Glissante

Pourquoi : Lorsque des suppressions multiples surviennent ou qu'une page est partiellement compactée, des cellules complètes se retrouvent dans l'espace non-alloué (entre la fin du tableau de pointeurs et le début de la zone de cellules).

Fonctionnement de l'Algorithme :

- **Accélération par saut de zéros (Zero-Run Fast Skip) :** L'espace non-alloué contient souvent des plages de zéros consécutifs (padding). L'algorithme avance immédiatement sur les séquences nulles supérieures à 1 octet, réduisant le temps d'analyse par 10 sur les grandes bases.
- **Validation à Deux Niveaux :**
 1. *Format Cellule* : Décodage du varint `payload_size`, validation de la plage `[2, 100 ext{ Mo}]`, lecture du varint `rowid`, et vérification que `L_{payload} = ext{header\_size} + \sum ext{type\_len}`.
 2. *Format Record Brut* : Si le header de cellule est corrompu, tentative directe de lecture du *Record Header* en évaluant la plausibilité d'impression des colonnes texte et des limites de valeurs numériques.

2.3 Résurrection de Schémas après DROP TABLE

Pourquoi : Lors de l'exécution d'une commande `DROP TABLE`, SQLite accomplit deux actions : 1) Il supprime la ligne correspondante dans `sqlite_master` sur la Page 1 (la déplaçant dans l'espace non-alloué de la Page 1). 2) Il déplace toutes les pages de données de la table dans la *freelist*. Les outils traditionnels échouent à typer les données de la *freelist* car le schéma de la table n'existe plus dans le catalogue actif.

La Solution SQLite-Carver-Pro :

1. Durant l'analyse de la Page 1, le carver scanne spécifiquement les freeblocks et l'espace non-alloué pour détecter les lignes `sqlite_master` supprimées (identifiées par le type `"table"` et la présence de `CREATE TABLE`).
2. Le schéma sculpté est immédiatement reconstitué et promu dans le dictionnaire `self.recovered_schemas`.
3. Lorsque les pages de la *freelist* sont sculptées, un second passage de corrélation applique le schéma ressuscité : les colonnes génériques (`col_0`, `col_1`) sont automatiquement remplacées par leurs vrais noms et types d'origine (ex : `id`, `nom`, `prenom`, `code`).

2.4 Traitement des Tables WITHOUT ROWID

Spécificité Technique : Les tables déclarées avec `WITHOUT ROWID` n'utilisent pas de B-Tree Table classique (`0x0D`). Toutes leurs données sont stockées sous la forme de B-Tree Index (`0x0A`). Dans ce mode, il n'y a aucun RowID en tête de cellule : la clé primaire et l'ensemble des colonnes de données sont empaquetées ensemble au sein du record payload.

Gestion par l'Outil : SQLite-Carver-Pro cartographie la page racine de la table dans `table_root_map` avec le drapeau `is_without_rowid`. Lorsqu'une page de type `0x0A` correspond à cette racine, elle est traitée comme une table de données primaire active avec attribution de confiance 1.0.

2.5 Désencapsulation Binaire des Tables Virtuelles R-Tree

Problème : Le module spatial SQLite R-Tree stocke les géométries (bounding boxes) sous forme de BLOBs opaques dans une table d'ombre interne appelée `<nom>_node`. Aucun parseur standard ne peut afficher ces coordonnées.

Décodage Physique (`unpack_rtree_leaf_node`) :

- Octet 0 à 1 : `depth` (uint16_be). Les feuilles de l'arbre spatial ont obligatoirement `depth == 0`.
- Octet 2 à 3 : `n_entries` (uint16_be), nombre d'entrées contenues dans le nœud.
- Chaque entrée feuille mesure $8 + 8D$ octets (pour D dimensions) :
 - 8 octets : Identifiant entier signé 64 bits (int64_be, `rowid`).
 - $2D$ imes 4 octets : Coordonnées réelles codées en virgule flottante IEEE 754 32 bits (float32_be). Pour 2D : `minX, maxX, minY, maxY`.

L'outil unpacke ces structures à la volée et émet des enregistrements natifs assignés directement à la table spatiale virtuelle.

3. Moteur Diff Temporel des Transactions (WAL & Rollback)

Le moteur de différentiel transactionnel de SQLite-Carver-Pro permet d'établir une chronologie inaltérable des opérations survenues dans la base de données en exploitant le fichier journal WAL (*Write-Ahead Log*) ou Rollback Journal.

3.1 Découpage des Frames WAL & Reconstruction

En mode WAL, les modifications ne sont pas écrites directement dans la base mais accumulées séquentiellement dans le fichier `.db-wal` sous forme de frames de taille fixe :

```
[ WAL Header (32 B) ] [ Frame 1: Header (24 B) + Page Data ] [ Frame 2: Header (24 B) + Page Data ] ...
```

Chaque en-tête de frame de 24 octets contient :

- **Page Number (4 B)** : Le numéro de page de la base que cette frame met à jour.
- **Commit Counter (4 B)** : Si non nul, indique la fin d'une transaction commitée et la taille de la base à cet instant.
- **Salt-1 / Salt-2 (8 B)** : Valeurs aléatoires changeant à chaque réinitialisation du WAL pour garantir l'ordre des transactions.
- **Checksum (8 B)** : Deux sommes de contrôle cumulatives vérifiant l'intégrité de la frame.

3.2 Génération des Diffs de Colonnes (Avant / Après)

En comparant la version d'une ligne présente dans la base principale avec ses versions antérieures ou postérieures dans le WAL, SQLite-Carver-Pro reconstitue l'état exact des mutations :

- `INSERT` : Ligne présente dans la frame WAL mais absente de la base d'origine.
- `UPDATE` : Ligne modifiée avec calcul du différentiel précis colonne par colonne (ex : `solde: 1500 € → 200 €`).
- `DELETE` : Ligne présente dans une version antérieure mais absente de la frame finale ou de la base.

3.3 Analyse de la Mémoire Partagée (`.db-shm`)

Le fichier `.db-shm` sert d'index en mémoire partagée entre les différents processus lisant et écrivant dans le WAL. L'analyseur `ShmAnalyzer` inspecte l'en-tête de 48 octets pour extraire :

- Le numéro de séquence du dernier checkpoint (`aFrame[0]`).
- Le nombre de frames écrites lors du dernier backfill (`aFrame[1]`).
- Les verrous de lecteurs actifs (slots 0 à 7), révélant si des processus étaient en cours de lecture lors de la saisie du disque.

4. Décodeurs Forensiques Profonds & Chaîne de Garde

4.1 Microscopie des BLOBs Applicatifs

Les applications modernes stockent fréquemment des structures complexes sérialisées dans les colonnes BLOB. SQLite-Carver-Pro intègre 4 moteurs d'inspection réursive :

- **Apple Binary Plist (`bp1ist00`)** : Parseur natif extrayant les dictionnaires, tableaux, UUIDs et dates Cocoa sans dépendre du système d'exploitation macOS.
- **Google Protocol Buffers (`protobuf`)** : Parseur de wire-types dynamique sans schéma `.proto` compilé. Filtre rigoureusement les faux positifs sur les types `varint`, `fixed32`, `fixed64` et `length-delimited`.
- **Compression Zlib / Deflate** : Détection automatique du header zlib (`0x78 0x9C`, `0x78 0x01`, `0x78 0xDA`) et décompression à la volée.
- **Extraction de Fichiers Embarqués** : Détection des signatures magiques (PNG, JPEG, GIF, PDF, ZIP, SQLite) et extraction automatique sur disque avec `--dump-blobs`.

4.2 Normalisation des Horodatages Numériques

Le moteur `decode_timestamp` analyse chaque valeur numérique et identifie heuristiquement son référentiel temporel :

Format Temporel	Époque de Référence	Résolution	Plage Validée
Unix Epoch (Sec)	01/01/1970 00:00:00 UTC	Seconde (10 chiffres)	Années 1995 à 2038
Unix Millisecondes	01/01/1970 00:00:00 UTC	Milliseconde (13 chiffres)	Années 2000 à 2040
WebKit / Chrome / Safari	01/01/1601 00:00:00 UTC	Microseconde (17 chiffres)	Années 2000 à 2040
Windows FILETIME	01/01/1601 00:00:00 UTC	Intervalles de 100 ns (18 chiffres)	Années 2000 à 2040
Apple Cocoa Absolute Time	01/01/2001 00:00:00 UTC	Secondes flottantes ou microsecondes	Années 2001 à 2040
GPS Time	06/01/1980 00:00:00 UTC	Secondes GPS (sans leap seconds)	Années 1980 à 2040

4.3 Entropie de Shannon & Détection de Chiffrement

Pour chaque page et sur l'ensemble de la base, l'outil calcule l'entropie de Shannon :

$$H = - \sum (p_i \times \log_2 p_i)$$

Une page en texte clair ou semi-remplie présente une entropie comprise entre 1.5 et 5.5. Une entropie supérieure à **7.95** indique avec certitude un conteneur entièrement chiffré (ex : *SQLCipher*, *wxSQLite3*, ou *SEE* avec chiffrement AES-256-CBC) ou des flux compressés dissimulés.

4.4 Chaîne de Garde Cryptographique & Hachage des Preuves

Pour garantir l'intégrité probatoire et la recevabilité judiciaire selon la norme ISO/IEC 27037 :

- Chaque fichier physique (base, journal, WAL, SHM) est haché en SHA-256 et MD5 dès son ouverture.

- Chaque enregistrement extrait reçoit un hash cryptographique déterministe calculé sur l'ensemble de ses attributs physiques et logiques : `SHA-256(page_id || offset_in_page || source || matched_table || rowid || values)`.

5. Guide des Commandes CLI & Rapports Interactifs

5.1 Commandes d'Investigation CLI

```
# 1. Carving complet avec rapport HTML interactif et timeline WAL
sqlite-carver carve scelle.db --wal --html rapport.html --out preuves.json

# 2. Extraction isolée des modifications de lignes (Diffs Avant/Après)
sqlite-carver carve scelle.db --mutations-only --html mutations.html

# 3. Chasseur de pages brutes dans un dump RAM ou une image disque
sqlite-carver carve-raw dump_memoire.raw --workers 8 --html ram_report.html

# 4. Bilan détaillé de l'allocation physique et du slack forensique
sqlite-carver storage-breakdown scelle.db

# 5. Diagnostic de la mémoire partagée (.db-shm) et verrous
sqlite-carver analyze-shm scelle.db-shm

# 6. Évaluation d'entropie et détection de bases chiffrées (SQLCipher)
sqlite-carver analyze-encryption scelle.db

# 7. Recherche forensique récursive de mots-clés et tokens
sqlite-carver search scelle.db "session_token_xyz"
```

5.2 Le Rapport HTML Interactif (Air-Gapped)

Le rapport généré avec `--html` est une application web autonome ne nécessitant aucune connexion Internet :

- **Bandeau de Répartition du Stockage** : Synthèse en 6 cartes (Données Actives, Freeblocks, Espace Non-Alloué, Slack Fragmenté, Freelist, et Espace Réservé par page).
- **Moteur de Recherche Instantané** : Indexation locale permettant de filtrer des millions d'enregistrements en temps réel (60 FPS).
- **Vue Dédoublement WAL** : Timeline chronologique des transactions avec mise en valeur colorée des colonnes altérées.
- **Bilingue Immédiat** : Bascule instantanée Français / Anglais sans rechargement.

Distribution Autonome Portable

SQLite-Carver-Pro est fourni sous la forme d'un exécutable unique 64 bits (`sqlite-carver-v1.7.0.exe`) fonctionnant directement sur les stations d'analyse forensique isolées du réseau sans nécessiter l'installation de Python.